

PUCRS — ENGENHARIA DE SOFTWARE — 4º SEMESTRE

SysOp

Simulador de Sistema

Operacional

Documentação Técnica Completa — Trabalho T1 (A + B)

Disciplina: Sistemas Operacionais — Prof. Fernando Dotti

Partes implementadas: T1-A (Gerenciador de Memória) + T1-B (Gerenciador de Processos)

Linguagem: Java (sem frameworks externos)

Memória RAM simulada: 1.024 palavras / 64 frames de 16 palavras cada

Processos: criação, carregamento em frames, execução com paginação física, remoção

Sumário

1. Visão Geral do Projeto
2. Arquitetura Geral
3. Camada de Hardware Simulado (`Sistema.java`)
 - 3.1 Memória • 3.2 Palavra (Word) • 3.3 CPU e ciclo de instrução • 3.4 Opcodes • 3.5 Interrupções • 3.6 Syscalls
4. Gerenciador de Memória Física (`T1-A`)
 - 4.1 Frame • 4.2 Page • 4.3 MemoryManager
5. Gerenciador de Processos (`T1-B`)
 - 5.1 PCB • 5.2 ProcessStatus • 5.3 ProcessManager
6. Tradução de Endereços Lógicos em Físicos
7. Interface de Linha de Comando (`Main.java`)
8. Programas de Teste
9. Escalonamento (T1-C — visão futura)
10. Fluxo Completo de uma Execução

1. Visão Geral do Projeto

O **SysOp** é um simulador educacional de sistema operacional desenvolvido em Java puro para a disciplina de Sistemas Operacionais da PUCRS. O objetivo é demonstrar, de forma didática e executável, os conceitos fundamentais de um SO real: gerenciamento de memória com paginação física, criação e ciclo de vida de processos, execução de programas na CPU simulada e tratamento de interrupções.

O trabalho é dividido em partes incrementais:

Parte	Nome	O que adiciona	Status
T1-A	Gerenciador de Memória (GM)	Paginação física: alocar/liberar frames, carregar programa em frames não-contíguos	Concluído
T1-B	Gerenciador de Processos (GP)	PCB, fila de prontos, criar/remover processo, exec, execAll, CLI	Concluído
T1-C	Escalonador (Scheduler)	Round-Robin com quantum, interrupção de timer, múltiplos processos concorrentes	Em andamento

PRINCÍPIO CENTRAL

Os programas de usuário usam **endereços lógicos contíguos** (0, 1, 2, ...). O sistema operacional é responsável por mapear esses endereços para **frames físicos** espalhados pela RAM, usando a tabela de páginas do processo.

PONTOS PARA APRESENTAR NESTA SEÇÃO

- Contextualizar o problema: por que um SO precisa de gerenciamento de memória e de processos?
- Mostrar que o projeto simula uma máquina real em miniatura: CPU, RAM, SO, programas.
- Ressaltar que *não há frameworks* — tudo construído a partir do zero em Java.
- Apresentar o diagrama de camadas (hardware → SO → usuário).

POSSÍVEIS PERGUNTAS DO PROFESSOR

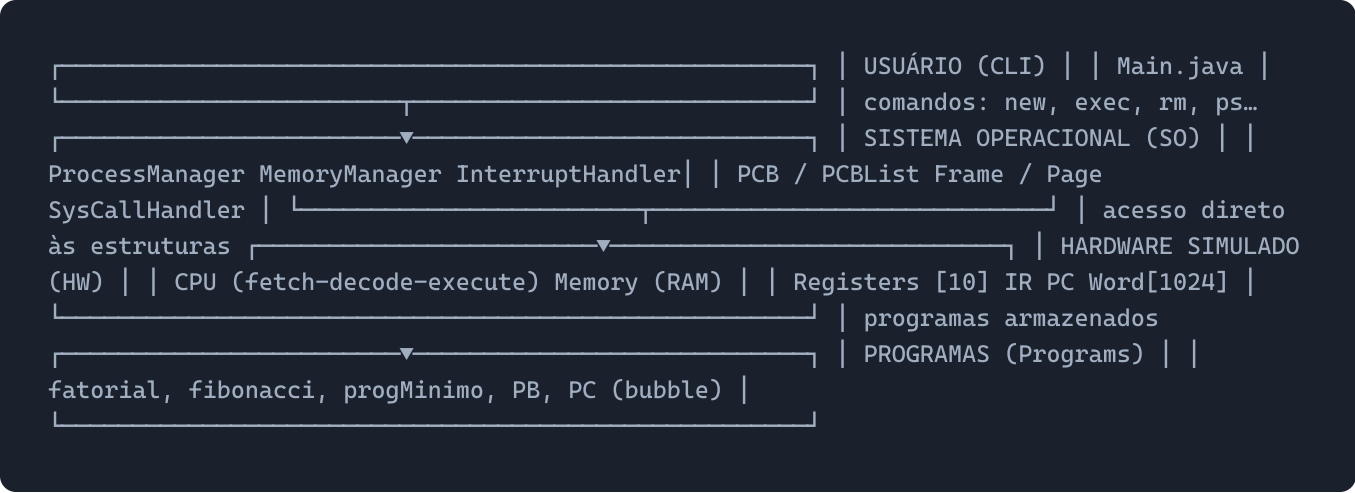
- Por que simular em Java e não usar C?*
Java oferece coleta de lixo e tipos seguros, reduzindo erros de ponteiro que desviariam o foco do aprendizado. O comportamento do SO é modelado nas classes, não na linguagem.

- *O que diferencia T1-A de T1-B?*

T1-A lida exclusivamente com a gestão de frames físicos (alocação/liberação). T1-B adiciona a abstração de processo: PCB, fila de prontos, ciclo de vida e execução da CPU em contexto de processo.

2. Arquitetura Geral

O sistema é organizado em quatro camadas bem definidas, cada uma contida em seu próprio pacote Java:



Pacotes e responsabilidades

Pacote / Arquivo	Responsabilidade
system.Sistema	Toda a simulação de hardware: CPU, memória, HW, SO, programas — uma única classe com inner classes
memoria.MemoryManager	Controla quais frames estão livres/ocupados; aloca e libera conjuntos de frames (páginas)
memoria.Frame	Representa um frame físico (índice, endereço início/fim, flag de ocupação)
memoria.Page	Mapeia uma página lógica ao seu frame físico correspondente
ProcessManager.PCB	Process Control Block — estado de um processo (PID, status, PC salvo, registradores, páginas)
ProcessManager.ProcessStatus	Enum de estados do processo
ProcessManager.ProcessManager	Cria, executa e remove processos; mantém a fila de prontos
Main	Shell interativo (CLI) que recebe comandos do usuário

PONTOS PARA APRESENTAR NESTA SEÇÃO

- Mostrar o diagrama de camadas e explicar o fluxo de uma chamada: usuário digita `new fatorial` → Main → ProcessManager → MemoryManager → memória física.
- Destacar que o hardware é completamente *simulado em software*, mas o código do SO não sabe disso — ele interage com a CPU e a memória como se fossem reais.
- Apontar a separação de responsabilidades: CPU não sabe de processos, MemoryManager não sabe de CPU.

3. Camada de Hardware Simulado (Sistema.java)

Toda a simulação de hardware reside em `src/system/Sistema.java`. A classe usa *inner classes* para agrupar logicamente os componentes de hardware, mantendo acesso privilegiado entre eles sem expor detalhes ao SO.

3.1 Memória (Memory)

A RAM é modelada como um vetor de `Word`. O tamanho padrão é **1.024 palavras**. Cada posição é inicializada com o opcode `___` (nulo), indicando memória não usada.

```
public class Memory {
    public Word[] pos; // pos[i] é a posição i da memória

    public Memory(int size) {
        pos = new Word[size];
        for (int i = 0; i < pos.length; i++) {
            pos[i] = new Word(Opcode.___, -1, -1, -1); // inicializa como vazia
        }
    }
}
```

DETALHE IMPORTANTE

A memória *não distingue* código de dados — toda posição é um `Word`. O opcode `DATA` sinaliza que aquela posição contém um dado (não deve ser executada como instrução). Isso reflete a arquitetura de Von Neumann.

3.2 Palavra (Word)

Cada célula de memória é uma **instrução de 4 campos**. Essa estrutura unificada permite que a CPU leia tanto instruções quanto dados do mesmo array.

```
public static class Word {
    public Opcode opc; // operação a realizar
    public int ra;     // registrador destino (ou primeiro operando)
    public int rb;     // registrador fonte (ou segundo operando)
    public int p;      // parâmetro imediato (endereço, constante ou dado)

    public Word(Opcode _opc, int _ra, int _rb, int _p) {
```

```

        opc = _opc; ra = _ra; rb = _rb; p = _p;
    }
}

```

Exemplo: `new Word(Opcode.LDI, 0, -1, 7)` representa a instrução `r[0] ← 7`. Campos não usados recebem `-1`.

3.3 CPU e Ciclo de Instrução

A CPU implementa o ciclo clássico **Fetch** → **Decode** → **Execute** → **Verificar Interrupção**. Ela possui 10 registradores de uso geral, PC, IR e uma flag de interrupção.

```

public class CPU {
    private int pc;           // Program Counter (endereço lógico atual)
    private Word ir;          // Instruction Register
    private int[] reg;        // 10 registradores de uso geral (reg[8] e reg[9] para
I/O)
    private Interrupts irpt;  // sinaliza interrupção durante a execução

    private int pageSize;     // tamanho de página (para tradução de endereços)
    private PCB currentProcess; // processo em execução (para acessar tabela de página
s)

    public void run(PCB runningProcess, boolean isExecAll) {
        this.currentProcess = runningProcess;
        System.arraycopy(runningProcess.savedRegisters, 0, reg, 0, reg.length); // res
taura contexto
        Integer cicleLimit = 5; // quantum: máximo de instruções por rodada
        Integer cicle = 0;
        cpuStop = false;

        while (!cpuStop) {
            // --- Preempção por quantum (apenas em execAll) ---
            if (isExecAll && cicle ≥ cicleLimit) {
                sysCall.stop();
                cpuStop = true;
                System.arraycopy(reg, 0, runningProcess.savedRegisters, 0, reg.lengt
h); // salva contexto
                processManager.pcbReadyList.add(runningProcess); // retorna à fila
                break;
            }

            // --- FASE 1: FETCH ---
            if (legal(pc)) {
                ir = m[translate(pc)]; // busca instrução: traduz PC lógico → físic
o
                runningProcess.processPc = pc; // salva PC no PCB
            }
        }
    }
}

```

```

        // --- FASE 2/3: DECODE + EXECUTE ---
        switch (ir.opc) {
            case LDI: reg[ir.ra] = ir.p; pc++; break;
            case ADD: reg[ir.ra] = reg[ir.ra] + reg[ir.rb]; testOverflow(reg
[ir.ra]); pc++; break;
            case STD:
                if (legal(ir.p)) { m[translate(ir.p)].p = reg[ir.ra]; pc++; }
                break;
            case STOP: sysCall.stop(); cpuStop = true; break;
            // ... demais opcodes
        }
    }

    // --- FASE 4: VERIFICAR INTERRUPÇÃO ---
    if (irpt != Interrupts.noInterrupt) {
        ih.handle(irpt);
        cpuStop = true;
    }
    cycle++;
}

if (!interruption) runningProcess.status = ProcessStatus.FINISHED;
}
}

```

QUANTUM DE 5 INSTRUÇÕES

O limite de 5 ciclos só é aplicado quando `isExecAll = true`. No modo `exec` (processo único), a CPU roda até o processo terminar, encontrar erro ou chamar `STOP`.

3.4 Set de Instruções (Opcodes)

Categoria	Instruções	Descrição
Especiais	DATA, ____	DATA = posição de dado (não executável); ____ = posição vazia

Desvios	JMP, JMPI, JMPIG, JMPIIL, JMPIE, JMPIGK, JMPILK, JMPIEK, JMPIGT, JMPIM, JMPIGM, JMPILM, JMPIEM	Salto condicional e incondicional; sufixo K = destino imediato, M = destino em memória
Aritméticas	ADDI, SUBI, ADD, SUB, MULT	Operações inteiras com detecção de overflow (-32767 a 32767)
Movimentação	LDI, LDD, STD, LDX, STX, MOVE	Carga de imediato, carga/armazenamento em memória (direto e indireto por registrador)
Sistema	SYSCALL, STOP	SYSCALL = chamada de sistema (I/O); STOP = encerra processo

3.5 Tratamento de Interrupções

O mecanismo de interrupção é sinalizado pela flag `irpt` dentro do ciclo da CPU. Ao fim de cada instrução, a CPU verifica se uma interrupção foi gerada e desvia para o `InterruptHandling` do SO.

```
public enum Interrupts {
    noInterrupt,          // nenhuma interrupção pendente
    intEnderecoInvalido,  // acesso fora do espaço do processo
    intInstrucaoInvalida, // opcode DATA sendo executado, ou opcode desconhecido
    intOverflow           // resultado aritmético fora da faixa [-32767, 32767]
}

// Verificação de endereço lógico válido:
private boolean legal(int e) {
    if (e ≥ 0 && currentProcess ≠ null && e < currentProcess.pages.length * pageSiz
e) {
        return true;
    } else {
        irpt = Interrupts.intEnderecoInvalido;
        return false;
    }
}
```

PONTOS PARA APRESENTAR NESTA SEÇÃO

- Demonstrar ao vivo o ciclo fetch-execute com o modo `traceOn` ativado — a CPU imprime cada instrução e o estado dos registradores.
- Explicar por que `legal()` usa o espaço lógico do processo e não o endereço físico diretamente.
- Mostrar a tabela de opcodes e ressaltar o prefixo **K** (imediato) vs **M** (indireto via memória).
- Falar sobre overflow aritmético: a CPU simula um inteiro de 16 bits (-32767 a 32767).

POSSÍVEIS PERGUNTAS DO PROFESSOR

- *O que acontece quando a CPU acessa um endereço fora dos limites do processo?*

A função `legal()` sinaliza `intEnderecoInvalido`. Ao fim do ciclo, o `InterruptHandling` é acionado e a execução do processo é interrompida.
- *Por que o overflow é tratado como interrupção e não como exceção Java?*

Para simular fielmente o comportamento de hardware. Em processadores reais, overflow dispara uma interrupção que o SO pode tratar — ignorar, encerrar o processo ou sinalizar ao programa.
- *Como funciona o SYSCALL de I/O?*

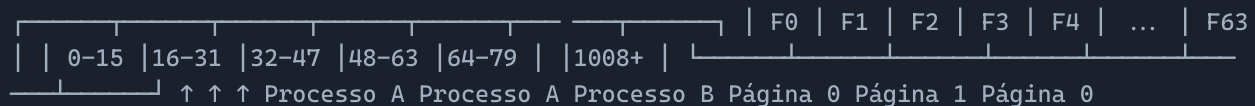
O programa carrega em `reg[8]` o tipo (1=leitura, 2=escrita) e em `reg[9]` o endereço lógico do dado. A instrução SYSCALL desvia para `SysCallHandling.handle()`, que imprime ou lê o valor.
- *Por que o PC é mantido como endereço lógico, não físico?*

O processo "pensa" em endereços lógicos. O PC representa a posição no espaço de endereçamento virtual do processo. A tradução para físico acontece no momento do fetch, dentro de `translate(pc)`.

4. Gerenciador de Memória Física — T1-A

O pacote `memoria` implementa a paginação física. A memória é dividida em **frames** de tamanho fixo (padrão: 16 palavras). Processos recebem um conjunto de frames — não necessariamente contíguos — que formam suas **páginas**.

Memória Física (1024 words, 64 frames de 16 words cada)



4.1 Frame (Frame.java)

Um frame representa um bloco físico de memória. É a unidade básica de alocação.

```
public class Frame {
    public static int indexCounter = 0; // contador global de frames
    public int index; // número único do frame (0 a 63)
    public int start; // endereço físico de início (ex: frame 2 → start=32)
    public int end; // endereço físico de fim (ex: frame 2 → end=47)
    public boolean isOccupied; // true se alocado a algum processo

    public Frame(int start, int end) {
        this.start = start;
        this.end = end;
        this.index = indexCounter++;
        // isOccupied inicializa false por padrão
    }
}
```

Fórmula: frame de índice `f` ocupa posições físicas `[f × pageSize, (f+1) × pageSize - 1]`.

4.2 Página (Page.java)

Uma `Page` associa uma página lógica do processo a um frame físico. É através da tabela de páginas (array `PCB.pages[]`) que a CPU realiza a tradução de endereços.

```

public class Page {
    public Frame frame; // referência ao frame físico correspondente
    Word[] words;       // cache das words (não essencial para execução)

    public Page(Frame frame, int size) {
        this.frame = frame;
        this.words = new Word[size];
    }
}

```

4.3 Gerenciador de Memória (MemoryManager.java)

O `MemoryManager` mantém um vetor `frameControl[]` com todos os frames da memória. Ele é a única entidade autorizada a marcar frames como ocupados ou livres.

```

public class MemoryManager {
    private int pageSize;
    private Memory memory;
    private Frame[] frameControl; // um Frame para cada bloco físico de pageSize words

    public MemoryManager(Memory memory, int pageSize) {
        this.memory = memory;
        this.pageSize = pageSize;
        this.frameControl = new Frame[memory.pos.length / pageSize]; // 1024/16 = 64
        initializeFrameControl();
    }

    private void initializeFrameControl() {
        for (int i = 0; i < frameControl.length; i++) {
            // Frame i ocupa [i*pageSize, (i+1)*pageSize - 1]
            frameControl[i] = new Frame(i * pageSize, ((i + 1) * pageSize) - 1);
        }
    }
}

```

Alocação: `allocate(nWords)`

Calcula quantos frames são necessários para `nWords` palavras e retorna um array de `Page` com os frames alocados. Usa divisão com teto (*ceiling division*) para garantir que sobras caibam em um frame extra.

```

public Page[] allocate(int wordsSize) {
    int numPages = (wordsSize + pageSize - 1) / pageSize; // teto: ex. 20 words / 16 =
    2 frames
}

```

```

    Frame[] available = getAvailableFrames(); // filtra frames com isOccupied == false
    if (numPages > available.length) {
        return null; // sem memória disponível
    }

    Page[] pages = new Page[numPages];
    for (int i = 0; i < numPages; i++) {
        available[i].isOccupied = true; // marca como ocupado
        pages[i] = new Page(available[i], pageSize);
    }
    return pages;
}

public Frame[] getAvailableFrames() {
    return Arrays.stream(frameControl)
        .filter(frame → !frame.isOccupied)
        .toArray(Frame[]::new);
}

```

Liberação: deallocate(pages[])

Libera todos os frames de um processo, zerando a memória física e marcando os frames como disponíveis.

```

public void deallocate(Page[] pages) {
    for (Page page : pages) {
        page.words = null;
        page.frame.isOccupied = false;
        // limpa a memória física do frame
        for (int i = page.frame.start; i ≤ page.frame.end; i++) {
            memory.pos[i] = new Word(Opcode.___, -1, -1, -1);
        }
    }
}

```

ESTRATÉGIA DE ALOCAÇÃO: FIRST-FIT IMPLÍCITO

O `getAvailableFrames()` retorna frames livres em ordem de índice. O `allocate()` usa sempre os primeiros da lista — comportamento equivalente ao **First-Fit**. Em cenários com muita fragmentação externa, isso poderia ser substituído por Best-Fit.

PONTOS PARA APRESENTAR NESTA SEÇÃO

- Demonstrar visualmente a fragmentação: criar dois processos, remover o primeiro, criar um terceiro maior — os frames do primeiro ficam disponíveis para reutilização.
- Mostrar o comando `dumpM` que exibe a quantidade de frames livres e ocupados.
- Explicar por que usamos paginação em vez de segmentação ou alocação contígua.
- Calcular ao vivo: programa com 20 instruções → $\text{ceil}(20/16) = 2$ frames necessários.

POSSÍVEIS PERGUNTAS DO PROFESSOR

- Por que a divisão usa $(\text{wordsSize} + \text{pageSize} - 1) / \text{pageSize}$ e não simplesmente $\text{wordsSize} / \text{pageSize}$?

A divisão inteira trunca para baixo. Se o programa tem 17 words e o frame tem 16, $17/16 = 1$ (errado — faltaria 1 word). A fórmula de teto garante que o resto seja acomodado: $(17+15)/16 = 2$ frames.
- O que acontece se não há frames livres suficientes?

`allocate()` retorna `null`. O `ProcessManager` detecta isso, imprime mensagem de erro e a criação do processo falha sem corromper o estado do sistema.
- Por que os frames são zerados na liberação?

Para evitar vazamento de informação entre processos — um processo novo não deve ver dados do processo anterior. Em sistemas reais, isso é obrigatório por segurança.
- Qual a diferença entre fragmentação interna e externa nesse modelo?

Fragmentação interna: o último frame de um processo pode estar parcialmente vazio (ex: 3 words usadas de 16). Fragmentação externa não existe em paginação pura — qualquer conjunto de frames livres pode ser usado.

5. Gerenciador de Processos — T1-B

5.1 Process Control Block (PCB.java)

O PCB é a **representação interna de um processo** dentro do SO. Contém tudo que o sistema precisa saber sobre o processo para pausar e retomar sua execução.

```
public class PCB {
    private static int idCounter = 0;    // PID auto-incremento global

    public Integer id;                    // PID único do processo
    public ProcessStatus status;         // estado atual (CREATED, READY, EXECUTING...)
    public Frame[] frames;               // frames físicos alocados para este processo
    public Page[] pages;                 // tabela de páginas (mapeamento lógico → físico)

    public Integer processPc;             // PC salvo (endereço lógico para retomar execução)

    public int[] savedRegisters = new int[10]; // contexto dos 10 registradores (context switch)

    private String programName;          // nome do programa (para exibição)

    PCB() {
        id = idCounter++;
        status = ProcessStatus.CREATED;
        programName = "unknown";
    }

    public void setFrames(Frame[] programFrames) {
        frames = programFrames;
        processPc = 0; // PC inicia em 0 (endereço lógico 0 = início do programa)
    }

    public void setPages(Page[] programPages) {
        pages = programPages;
    }
}
```

CONTEXT SWITCH VIA SAVEDREGISTERS

Quando um processo é preemptado (quantum esgotado em `execAll`), a CPU copia `reg[]` para `pcb.savedRegisters[]` antes de liberar a CPU. Quando o processo retoma, os registradores são restaurados de

`savedRegisters[]` antes de iniciar o ciclo.

5.2 Estados do Processo (`ProcessStatus.java`)

```
public enum ProcessStatus {
    CREATED,    // PCB criado, ainda não entrou na fila de prontos
    READY,      // na fila de prontos, aguardando CPU
    EXECUTING,  // usando a CPU neste momento
    PAUSE,      // interrompido por preempção (quantum), voltará à fila
    FINISHED    // terminou (STOP ou erro fatal) - será removido
}
```

```
criar processo → CREATED | add à fila | de prontos ▼ READY
← | | escalonador | | preempção seleciona ▼ | (quantum) EXECUTING
→ | | STOP/erro | ▼ FINISHED
```

5.3 Gerenciador de Processos (`ProcessManager.java`)

O `ProcessManager` é o coração do T1-B. Ele orquestra todos os aspectos do ciclo de vida de processos.

Criação de processo: `createProcess(name)`

```
public boolean createProcess(String programName) {
    Word[] programImage = programs.retrieveProgram(programName); // busca programa em
    "disco"
    if (programImage == null) {
        System.out.println("[PM] Erro: Programa '" + programName + "' não encontrad
o.");
        return false;
    }
    return createProcessFromImage(programImage, programName);
}

private boolean createProcessFromImage(Word[] programImage, String programName) {
    // 1. Alocar páginas (e frames) no MemoryManager
    Page[] programPages = memoryManager.allocate(programImage.length);
    if (programPages == null) return false; // sem memória

    // 2. Extrair os frames das páginas
    Frame[] frames = getFramesPromPages(programPages);
}
```

```

// 3. Criar e configurar o PCB
PCB currentProgramPCB = new PCB();
currentProgramPCB.setProgramName(programName);
currentProgramPCB.setFrames(frames); // seta frames e inicializa processPc = 0
currentProgramPCB.setPages(programPages);

// 4. Copiar o código para os frames físicos
loadProgramInMemory(currentProgramPCB, programImage);

// 5. Adicionar à fila de prontos
pcbReadyList.add(currentProgramPCB);
System.out.println("[PM] Processo criado: PID=" + currentProgramPCB.id);
return true;
}

```

Carregamento na memória: `loadProgramInMemory()`

Copia as words do programa para os frames físicos em ordem, respeitando os limites de cada frame. Isso transforma o programa (que usa endereços lógicos contíguos) em código espalhado pela RAM física.

```

private void loadProgramInMemory(PCB currentProgramPCB, Word[] programImage) {
    int frameSize = currentProgramPCB.frames[0].end - currentProgramPCB.frames[0].start + 1; // 16
    int countWords = 0;

    for (int i = 0; i < currentProgramPCB.frames.length; i++) { // para cada frame
        for (int j = 0; j < frameSize; j++) { // para cada posição no frame
            if (countWords < programImage.length) {
                memory.pos[currentProgramPCB.frames[i].start + j] = programImage[countWords];
                countWords++;
            }
            // posições restantes no último frame ficam com Word(____, -1, -1, -1)
        }
    }
}

```

Execução: `executeProcess(pid, isExecAll)`

```

public void executeProcess(int pid, boolean isExecAll) {
    PCB pcb = getProcess(pid);
    if (pcb == null) { System.out.println("[PM] Processo não encontrado."); return; }
}

```

```

    running = pcb;
    pcb.status = ProcessStatus.EXECUTING;

    int startAddress = pcb.processPc; // retoma de onde parou (0 na primeira vez)
    cpu.setContext(startAddress);      // seta PC da CPU (endereço lógico)

    pcbReadyList.remove(pcb);          // remove temporariamente da fila durante execuã
o
    cpu.run(running, isExecAll);       // executa (com ou sem quantum)

    if (pcb.status == ProcessStatus.FINISHED) {
        memoryManager.deallocate(running.pages); // libera memória automaticamente
        System.out.println("[PM] Processo PID=" + pid + " finalizado.");
        running = null;
    } else {
        System.out.println("[PM] Processo PID=" + pid + " interrompido.");
        // processo voltou para pcbReadyList dentro da CPU (preempção)
    }
}

// execAll: roda cada processo na fila até todos terminarem
public void execAll() {
    while (!pcbReadyList.isEmpty()) {
        executeProcess(pcbReadyList.get(0).id, true);
    }
}

```

Remoção: deallocateProcess(id)

```

public void deallocateProcess(int id) {
    PCB currentPCB = pcbReadyList.stream()
        .filter(a → a.id == id).findFirst().get(); // busca o processo

    memoryManager.deallocate(currentPCB.pages); // libera frames
    pcbReadyList.remove(currentPCB);           // remove da fila
    deallocateProcessInMemory(currentPCB);      // zera a memória física
}

private void deallocateProcessInMemory(PCB currentPCB) {
    int frameSize = currentPCB.frames[0].end - currentPCB.frames[0].start + 1;
    for (int i = 0; i < currentPCB.frames.length; i++) {
        for (int j = 0; j < frameSize; j++) {
            memory.pos[currentPCB.frames[i].start + j] = new Word(Opcode.___, -1, -1,
-1);
        }
    }
}

```

```
}  
}
```

NOTA DE IMPLEMENTAÇÃO

A memória é limpa tanto em `MemoryManager.deallocate()` quanto em `deallocateProcessInMemory()`. Há uma sobreposição — na prática, uma das duas é redundante. Isso não causa erro, apenas limpeza dupla.

PONTOS PARA APRESENTAR NESTA SEÇÃO

- Mostrar o diagrama de estados do processo e percorrer cada transição com exemplos de comandos.
- Demonstrar ao vivo: `new fatorial` → `ps` (vê status CREATED/READY) → `exec 0` → processo vai para FINISHED.
- Enfatizar o salvamento de contexto: executar `execAll` com dois processos e mostrar que cada um retoma do ponto correto.
- Mostrar que o PCB é o "passaporte" do processo: sem ele, a CPU não saberia por onde continuar.

POSSÍVEIS PERGUNTAS DO PROFESSOR

- *O que é um PCB e quais informações mínimas ele precisa conter?*

PCB = Process Control Block. É a estrutura de dados que o SO usa para representar um processo. Mínimo necessário: PID, estado, PC salvo, registradores salvos, tabela de páginas (ou ponteiro para ela) e informações de scheduling.

- *Como o processo é "pausado" e "retomado" corretamente?*

Pausa: a CPU salva `reg[]` em `pcb.savedRegisters[]` e o PC lógico em `pcb.processPc`.

Retomada: `cpu.setContext(pcb.processPc)` restaura o PC, e

`System.arraycopy(savedRegisters, reg)` restaura os registradores.

- *O que acontece se tentarmos fazer `rm` de um processo em execução?*

A implementação atual busca o processo apenas na `pcbReadyList`. Um processo em execução está em `running`, fora da lista, então `findFirst().get()` lançaria `NoSuchElementException`. Uma melhoria seria verificar `running` também.

- *Por que o processo é removido da fila de prontos antes de executar?*

Para evitar que o mesmo processo seja selecionado novamente durante a execução (especialmente em `execAll`). Ele só retorna à fila se for preemptado — caso contrário, termina e é descartado.

6. Tradução de Endereços Lógicos em Físicos

Este é o núcleo conceitual do trabalho — a ponte entre o que o programa enxerga (endereços lógicos contíguos) e o que a RAM física possui (frames espalhados).

6.1 Fórmula de Tradução

```
Endereço lógico → número da página + offset → endereço físico  
endereçoLógico = 34  
pageSize = 16 página = 34 / 16 = 2 (índice da página na tabela) offset = 34 % 16 = 2  
(posição dentro do frame) frame físico da página 2 = pages[2].frame.start = 48 (ex:  
processo em frames 0, 1, 2) endereçoFísico = 48 + 2 = 50
```

6.2 Implementação na CPU

```
// Na classe CPU (Sistema.java):  
public int translate(int logicalAddr) {  
    int page = logicalAddr / pageSize;           // qual página lógica  
    int offset = logicalAddr % pageSize;         // posição dentro da página  
    return currentProcess.pages[page].frame.start + offset; // endereço físico  
}  
  
// Uso no fetch da instrução:  
ir = m[translate(pc)]; // busca instrução no endereço físico correspondente ao PC lógico  
  
// Uso em LDD (load direto):  
case LDD: // Rd ← [A] - A é endereço lógico  
    if (legal(ir.p)) {  
        reg[ir.ra] = m[translate(ir.p)].p; // traduz antes de acessar a memória  
        pc++;  
    }  
    break;
```

6.3 Verificação de Acesso (legal())

Antes de qualquer tradução, a CPU verifica se o endereço lógico está dentro dos limites do processo. Isso previne que um processo acesse memória de outro processo.

```
private boolean legal(int e) {
    // e é um endereço lógico
    // o espaço total do processo = número de páginas × tamanho da página
    if (e ≥ 0 && currentProcess ≠ null && e < currentProcess.pages.length * pageSiz
e) {
        return true;
    } else {
        irpt = Interrupts.intEnderecoInvalido; // dispara interrupção de segmentation
fault
        return false;
    }
}
```

6.4 Exemplo Completo

Processo com programa de 18 words, pageSize = 16. Alocação obtém os frames 3 e 7 (endereços físicos 48–63 e 112–127).

Endereço Lógico	Página	Offset	Frame Físico	Endereço Físico
0	0	0	3 (start=48)	48
15	0	15	3 (start=48)	63
16	1	0	7 (start=112)	112
17	1	1	7 (start=112)	113

PONTOS PARA APRESENTAR NESTA SEÇÃO

- Este é o ponto central do trabalho — dedicar tempo suficiente para explicar a fórmula de tradução.
- Mostrar com um diagrama desenhado à mão (ou no slide): programa contíguo no papel → frames espalhados na RAM.
- Conectar com o conceito de TLB (Translation Lookaside Buffer) em sistemas reais — aqui não implementamos TLB, então toda tradução é feita pela tabela de páginas.
- Demonstrar ao vivo com `traceOn` : a CPU mostra o PC lógico, e o `dump` mostra os frames físicos onde o código está.

POSSÍVEIS PERGUNTAS DO PROFESSOR

- *O que é a fragmentação interna nesse modelo e onde ela ocorre?*

No último frame de um processo: se o programa tem 17 words e o frame tem 16, o segundo frame tem apenas 1 word usada e 15 desperdiçadas. Esse desperdício = fragmentação interna. Frames intermediários não têm fragmentação.

- *Como a paginação garante proteção entre processos?*

Cada processo tem sua própria tabela de páginas (`PCB.pages[]`). A CPU usa apenas essa tabela para traduzir endereços. Um processo com páginas mapeadas para frames F0 e F1 nunca "enxerga" frames F2+ de outro processo, pois seu espaço lógico acaba no limite de suas páginas — qualquer acesso além disso é barrado por `legal()` .

- *Qual seria a complexidade de adicionar memória virtual (swap) a esse sistema?*

Precisaríamos adicionar: bit de validade na tabela de páginas, rotina de page fault, área de swap em disco (simulado), e algoritmo de substituição (LRU, Clock...). O projeto já inclui

`GerenciadorMemoriaVirtual.java` como extensão opcional com algoritmo Second Chance.

7. Interface de Linha de Comando (Main.java)

O `Main.java` é o **shell do sistema operacional simulado**. Inicializa todos os componentes e entra em um loop de comandos que permite ao usuário interagir com o sistema.

7.1 Inicialização

```
public static void main(String[] args) {
    int memorySize = 1024;
    int pageSize   = 16;

    sistema        = new Sistema(memorySize);           // cria HW: CPU + RAM
    memoryManager = new MemoryManager(sistema.hw.mem, pageSize); // divide RAM em 64 frames
    processManager = new ProcessManager(
        sistema.hw.mem, memoryManager,
        sistema.hw.cpu, sistema.progs);                // wires todos os subsistemas

    sistema.setProcessManager(processManager);
    sistema.setMemoryManager(memoryManager);

    sistema.hw.cpu.setAddressOfHandlers(sistema.so.ih, sistema.so.sc); // registra handlers de int/syscall
    sistema.hw.cpu.setUtilities(sistema.so.utils);
    sistema.hw.cpu.setPageSize(pageSize);
}
```

7.2 Comandos Disponíveis

Comando	Ação	Método chamado
new <prog>	Cria e carrega um processo na RAM	processManager.createProcess(name)
rm <pid>	Remove processo e libera memória	processManager.deallocateProcess(pid)
ps	Lista processos, PIDs e status	processManager.listProcesses()

<code>exec <pid></code>	Executa processo até terminar ou errar	<code>processManager.executeProcess(pid, false)</code>
<code>execAll</code>	Executa todos os processos na fila com quantum de 5	<code>processManager.execAll()</code>
<code>dump</code>	Imprime toda a RAM (todas as 1024 words)	<code>sistema.so.utils.dump(0, 1024)</code>
<code>dumpM</code>	Mostra frames livres e ocupados	<code>memoryManager.getAvailableFrames()</code>
<code>traceOn/Off</code>	Liga/desliga log detalhado da CPU	<code>sistema.hw.cpu.setDebug(true/false)</code>
<code>exit</code>	Encerra o sistema	—

PONTOS PARA APRESENTAR NESTA SEÇÃO

- Fazer uma demo ao vivo do fluxo completo: `new fatorial` → `ps` → `traceOn` → `exec 0` → observar a CPU executar instrução a instrução → `dumpM`.
- Mostrar que o shell é um *processo especial* que não usa a CPU simulada — ele roda na JVM. Isso corresponde ao "Thread Shell" do diagrama de T1-C.

8. Programas de Teste

Os programas são arrays de `Word[]` estáticos definidos na classe `Programs` dentro de `Sistema.java`. Eles representam "programas em disco" que podem ser carregados para a RAM.

Programa	Tamanho	O que faz	Usa SYSCALL?
<code>fatorial</code>	11 words	Calcula 7! com loop e registradores. Resultado em <code>pos[10]</code>	Não
<code>fatorialV2</code>	20 words	Versão aprimorada: trata negativos, escreve resultado com SYSCALL	Sim (saída)
<code>progMinimo</code>	14 words	Escreve valor 999 em 5 posições de memória. Teste básico de STD	Não
<code>fibonacci10</code>	30 words	Calcula série de Fibonacci com 10 termos, armazena em memória	Não
<code>fibonacci10v2</code>	33 words	Versão alternativa usando MOVE em vez de ADD+LDI	Não
<code>fibonacciREAD</code>	55 words	Lê tamanho da série via SYSCALL e gera N termos de Fibonacci	Sim (entrada e saída)
<code>PB</code>	16 words	Se número positivo → calcula fatorial; se negativo → retorna -1	Não
<code>PC</code>	54 words	Ordena vetor de 5 elementos com Bubble Sort	Não

8.1 Exemplo Anotado: `fatorial`

```
new Word(Opcode.LDI, 0, -1, 7), // pos 0: r0 = 7 (número a calcular)
new Word(Opcode.LDI, 1, -1, 1), // pos 1: r1 = 1 (acumulador do produto)
new Word(Opcode.LDI, 6, -1, 1), // pos 2: r6 = 1 (constante de decremento)
new Word(Opcode.LDI, 7, -1, 8), // pos 3: r7 = 8 (endereço do STD, destino do JMP)
new Word(Opcode.JMPIE, 7, 0, 0), // pos 4: se r0 == 0, PC = r7 (= 8), sai do loop
new Word(Opcode.MULT, 1, 0, -1), // pos 5: r1 = r1 * r0
new Word(Opcode.SUB, 0, 6, -1), // pos 6: r0 = r0 - r6 (r0--)
new Word(Opcode.JMP, -1, -1, 4), // pos 7: PC = 4 (volta ao teste)
new Word(Opcode.STD, 1, -1, 10), // pos 8: mem[10] = r1 (salva resultado)
```

```
new Word(Opcode.STOP,-1, -1, -1), // pos 9: fim  
new Word(Opcode.DATA,-1, -1, -1) // pos 10: slot de dado (resultado aqui)
```

Execução para $n=7$: $r1$ acumula $7 \rightarrow 42 \rightarrow 210 \rightarrow 840 \rightarrow 2520 \rightarrow 5040 = 5040 = 7!$

8.2 Exemplo: PC (Bubble Sort)

O mais complexo dos programas. Usa desvios indiretos via memória (`JMPILM` , `JMPIGM` , `JMPIEM`) para implementar os dois laços aninhados do Bubble Sort. Posições 96–99 armazenam os endereços de salto das chaves, tornando o código automodificável em espírito.

PONTOS PARA APRESENTAR NESTA SEÇÃO

- Mostrar o `fatorial` passo a passo com `traceOn` — é o mais didático para explicar o ciclo da CPU.
- Ressaltar que os programas usam apenas endereços lógicos — a tradução é invisível para eles.
- Mostrar que `fibonacciREAD` usa `SYSCALL` para I/O — conecta com o conceito de chamada de sistema.

9. Escalonamento — T1-C (Visão Futura)

O T1-C adiciona o escalonador Round-Robin completo. A base já está preparada no código atual:

- **Quantum de 5 instruções:** implementado no loop da CPU (`cicleLimit = 5`)
- **Salvamento de contexto:** `savedRegisters[]` e `processPc` já são salvos na preempção
- **Retorno à fila:** processo preemptado é re-inserido em `pcbReadyList`
- **execAll:** já implementa Round-Robin sequencial de forma simplificada

O que T1-C precisa adicionar:

1. **Interrupção de timer real:** disparada automaticamente após N ciclos, não apenas quando `isExecAll=true`
2. **Thread separada para CPU:** CPU em thread própria, bloqueada em semáforo, desbloqueada pelo escalonador
3. **Thread do escalonador:** acordada pela interrupção de timer, seleciona próximo processo
4. **Fila de bloqueados:** para processos aguardando I/O

```
Thread Shell | Thread CPU | Thread Escalonador
-----
| wait(semaSch) exec 0 | ← signal(semaCPU) | | new fatorial | wait(semaCPU)
| | → signal(semaSch) | ← acorda | save context | seleciona próximo | wait(semaCPU) |
signal(semaCPU) →
```

POSSÍVEIS PERGUNTAS DO PROFESSOR (T1-C)

- *O que é Round-Robin e por que usar quantum?*

Round-Robin é um algoritmo de escalonamento preemptivo que aloca uma fatia de tempo (quantum) igual para cada processo na fila circular. Quantum garante que nenhum processo monopolize a CPU — essencial para sistemas interativos.

- *Qual o impacto do tamanho do quantum no desempenho?*

Quantum muito pequeno: muitos context switches, overhead alto. Quantum muito grande: comportamento similar a FIFO, menor responsividade. O valor ótimo depende do tempo médio de burst de CPU dos processos.

- *Por que usar semáforos entre as threads?*

Para sincronização: a Thread CPU não pode executar sem que o escalonador a autorize, e o escalonador não pode selecionar um processo enquanto a CPU está executando. Semáforos garantem exclusão mútua e coordenação de ordem.

10. Fluxo Completo de uma Execução

Passo a passo do que acontece quando o usuário digita `new fatorial` seguido de `exec 0`:

```

Usuário digita: new fatorial | ⊢ Main.handleNew("fatorial") | ⊢
processManager.createProcess("fatorial") | ⊢ programs.retrieveProgram("fatorial") →
Word[11] | ⊢ memoryManager.allocate(11) | | ⊢ numPages = ceil(11/16) = 1 | | ⊢
getAvailableFrames() → [F0, F1, ..., F63] | | ⊢ F0.isOccupied = true | | ⊢ retorna
Page[1] { pages[0] → frame F0 (0-15) } | ⊢ new PCB() { id=0, status=CREATED, processPc=0
} | ⊢ pcb.setPages(pages) → pcb.pages[0].frame = F0 | ⊢ loadProgramInMemory: copia
Word[11] para mem.pos[0..10] | ⊢ pcbReadyList.add(pcb) → fila: [PID=0] Usuário digita:
exec 0 | ⊢ Main.handleExec(0) | ⊢ processManager.executeProcess(0, false) | ⊢ pcb =
pcbReadyList.get(0) // PID=0 | ⊢ running = pcb | ⊢ pcb.status = EXECUTING | ⊢
cpu.setContext(0) // PC lógico = 0 | ⊢ pcbReadyList.remove(pcb) | ⊢ cpu.run(pcb, false)
| ⊢ restaura savedRegisters → reg[] (todos 0) | | | ⊢ CICLO 1: pc=0 | | ⊢ legal(0)? →
0 < 1*16 ✓ | | ⊢ translate(0): página=0, offset=0, físico = F0.start+0 = 0 | | ⊢ ir =
mem[0] = LDI r0, 7 | | ⊢ execute: reg[0] = 7 | | ⊢ pc = 1 | | | ⊢ ... (ciclos 2 a 7:
loop fatorial) | | | ⊢ CICLO N: STOP encontrado | | ⊢ sysCall.stop() → imprime "SYSCALL
STOP" | | ⊢ cpuStop = true | | | ⊢ pcb.status = FINISHED (não foi interrupção) | ⊢
memoryManager.deallocate(pcb.pages) → F0 livre ⊢ running = null

```

SysOp — Simulador de Sistema Operacional — PUCRS Engenharia de Software

Disciplina: Sistemas Operacionais — Prof. Fernando Dotti — Trabalho T1 (A+B)

Documentação gerada para apresentação acadêmica — 2026